IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 6 March 2026, accepted 15 March 2026, date of publication 20 March 2026, date of current version 26 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3676344

RESEARCH ARTICLE

XGB-AirSched: A Machine Learning-Guided Multi-Objective Optimization Framework for Airport Slot Allocation

SAAD TALAL ALHARBI
Department of Computer Science, College of Computer Science and Engineering, Taibah University, Medina 42353, Saudi Arabia
e-mail: stharbi@taibahu.edu.sa

ABSTRACT The airport slot allocation problem poses a complex challenge in balancing operational efficiency, fairness, and delay minimization at congested airports. Researchers are turning to optimization-based approaches, and more recently machine learning (ML), to overcome the limitations of traditional slot allocation methods. However, ML and optimization have been applied only in sequence or separately, and conflict prediction has rarely been integrated into the evolutionary optimization process. Moreover, optimization frameworks have not been evaluated in settings that mimic real-time operations. This study introduces XGB-AirSched, a hybrid framework that integrates ML-based conflict prediction into a multiobjective evolutionary optimization process for intelligent slot assign-ment. The framework bridges computational optimization and air-traffic scheduling by combining discrete-event simulation with predictive optimization. By integrating an XGBoost classifier with the nondominated sorting ge-netic algorithm II, the search process avoids infeasible slot combinations while optimizing for minimal delay and equitable allocation. The resulting schedules were validated through discrete-event simulations to assess real-world feasibility. On real and synthetic datasets, XGB-AirSched achieved conflict-free allocations with improved con-vergence behavior, outperforming traditional optimization methods. In simulations on real-world-scale datasets, XGB-AirSched demonstrated practical readiness for deployment within airport scheduling systems.

INDEX TERMS Airport slot allocation, machine learning optimization, discrete-event simulation, XGBoost, NSGA-II, hybrid framework, multi-objective optimization, scheduling systems, air traffic scheduling.

I. INTRODUCTION

The airport slot allocation problem (ASAP) is a critical challenge faced by airports worldwide, especially in highly congested hubs where the demand for landing and take-off slots is significantly greater than the available capacity. An airport is similar to a busy theater with limited seats, with airlines as the audience and slots as tickets. These tickets need to be distributed fairly and efficiently, ensuring that the show runs smoothly without overcrowding or delays. In essence, ASAP is about deciding who gets to use the airport's resources, when, and under what conditions.

The problem centers on allocating a finite set of time slots—specific intervals during which an aircraft can land or take off—to competing airlines. Airlines often have conflicting preferences, such as peak travel times or connections to other flights, with allocation requiring a delicate balance. The challenge is further complicated by regulatory rules, operational constraints (such as noise restrictions or curfews), and the need to minimize delays and environmental impact. The objective is to satisfy airlines and ensure that airports operate efficiently, passengers experience minimal disruptions, and that the surrounding community is not overwhelmed by noise or pollution.

Traditionally, slot allocation has relied on historical precedence and rule-based systems such as the IATA Worldwide Airport Slot Guidelines (WASGs). These methods are administratively simple; however, they are not equipped to respond dynamically to shifting operational conditions or real-time demand surges. Over the past two decades, research has

The associate editor coordinating the review of this manuscript and approving it for publication was Rongbo Zhu.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

shifted toward optimization-based approaches, including integer programming and metaheuristics such as genetic algorithm (GA) and particle swarm optimization (PSO), which offer more flexible mechanisms for handling delay and fairness trade-offs.

Recently, machine learning (ML) has emerged as a complementary tool in the predictive modeling of air traffic dynamics. ML models can anticipate delay patterns, detect potential slot conflicts, or predict airline preferences. However, most studies apply ML and optimization sequentially or in isolation, with few integrating conflict prediction into the core of an evolutionary optimization process that guides allocation decisions. Although some optimization frameworks have demonstrated theoretical performance, they have not been validated in realistic operational settings via simulations [1], [2].

To address both gaps, this study introduces XGB-AirSched, a hybrid framework that embeds XGBoost-based conflict prediction into the nondominated sorting genetic algorithm II (NSGA-II) for multiobjective airport slot optimization. It further validates schedule feasibility through discrete-event simulation. The proposed approach optimizes slot allocation by minimizing operational conflicts, reducing average delay, and promoting an equitable distribution of slots among competing airlines.

This study makes the following contributions:

- ML and multiobjective optimization are integrated for intelligent airport slot assignment.
- A conflict-aware optimization strategy guided by XGBoost predictions is presented, enabling informed exploration of the solution space.
- Simulations using real and synthetic datasets assess the operational feasibility and robustness of optimized schedules.

In addition, this study also contributes to computational optimization methods with mathematically defined objectives and simulation-based evaluation, offering a reproducible algorithmic framework. By combining predictive modeling and simulation-based optimization, the proposed framework supports reproducible engineering solutions rather than purely theoretical models.

II. BACKGROUND AND RELATED WORK

ASAP has received significant attention because of the increasing air traffic demand and limited runway capacity at major airports. Traditionally, slot assignment has relied on rule-based or administrative procedures such as the IATA's WASG, which often use first-come-first-served (FCFS) policies or historical precedence [3], [4]. Although simple, these approaches struggle to accommodate operational dynamics, leading to inefficient slot usage and increased delays.

Mathematical and combinatorial optimization methods have been widely applied to improve allocation outcomes. Integer linear programming (ILP) and mixed integer programming formulations have been used to model slot allocation with fairness and delay minimization objectives [5], [6]. However, these methods often become computationally expensive as the problem scale increases, limiting real-time applicability.

Metaheuristic algorithms offer greater scalability and have gained popularity in the past two decades. GAs have been extensively studied for airport and air traffic scheduling because of their robustness in handling multiobjective and constrained optimization problems [7], [8]. PSO and ant colony optimization (ACO) have also been proposed for slot coordination and runway sequencing, resulting in improvements in delay and resource efficiency [9], [10].

Multiobjective evolutionary algorithms (MOEAs), such as NSGA-II and NSGA-III, have become the state-of-the-art methods for balancing delay, fairness, and conflict reduction in scheduling tasks [11], [12], [13]. These methods effectively explore the Pareto front and provide diverse trade-off solutions. Recent studies emphasize fairness as a central allocation objective, ensuring equitable slot distribution among competing airlines [14], [15].

The integration of ML into the optimization process has shown promise in recent years. ML models can provide conflict prediction, delay forecasting, or preference modeling to guide optimization more effectively [16], [17]. Gradient boosting frameworks such as XGBoost are particularly well suited for structured data tasks such as conflict likelihood estimation because of their accuracy and interpretability [18]. However, few studies have directly embedded such ML models into the core of the optimization loop for airport slot assignment.

Hybrid approaches that combine ML and MOEAs are emerging as promising directions. For example, Tang et al. [19] used ML to prioritize feasible neighborhoods in airside scheduling, whereas Hussein et al. [7] embedded predictive models to accelerate GA convergence. Nevertheless, most studies stop short of validating solutions in realistic operational settings.

Recently, reinforcement learning (RL) and deep learning have been increasingly explored in aviation resource scheduling tasks such as stand allocation, departure sequencing, and ground operations. For example, Liu et al. [20] applied RL with adaptive dynamic programming to departure scheduling, whereas Ouyang et al. [21] and Wang et al. [22] used deep RL for airport stand management and ground support vehicle scheduling, respectively. These approaches demonstrate the potential of end-to-end policy learning but require extensive scenario coverage and stable reward shaping for safety-critical deployment. By contrast, our framework embeds supervised conflict prediction to guide evolutionary search, offering a pragmatic balance between adaptability and regulatory transparency.

Simulation-based validation has become a crucial step in modern slot allocation research. Discrete-event simulation (DES) tools such as BlueSky and SimPy are used to evaluate the feasibility of schedules under dynamic constraints and stochastic arrival patterns [23], [24]. Such simulation-integrated frameworks offer more realistic

performance evaluations but remain underutilized in ML-driven approaches.

Beyond GA and NSGA-II, prior work has reported competitive results with PSO and ACO for slot coordination and runway sequencing. Our study focuses on testing whether ML-guided multiobjective search outperforms traditional evolutionary baselines; therefore, we selected a representative single-objective GA and multiobjective NSGA-II for controlled comparisons. PSO and ACO are covered in the literature review and are natural candidates for future benchmarking once the ML-guided component is finalized for production use.

In recent years, several new works have strengthened the case for hybrid and AI-driven approaches in slot allocation and airport operations. Li and Liu [25] explored multiairport system slot optimization with absolute fairness and rolling constraints for shared waypoints; Yang et al. [25] developed a tri-objective model that combines slot deviation and airline fairness with demand-side fairness considerations. Li et al. [27] applied deep RL to the real-time gate assignment problem, capturing dynamic uncertainties in flight operations. Liu et al. [28] leveraged RL to revise ground delay programs, integrating airborne and ground delay metrics with real operational data. These studies further confirm that integrating ML or RL with optimization yields more robust, fair, and adaptable systems, directly supporting our hybrid ML-guided NSGA-II framework.

This study contributes to this growing field by proposing XGB-AirSched, a hybrid framework that couples XGBoost-based conflict prediction with NSGA-II optimization and DES. Compared with prior methods, XGB-AirSched addresses three core challenges simultaneously: (1) predictive conflict avoidance via ML, (2) fairness and delay trade-off optimization via MOEAs, and (3) operational validation via simulation. This combination positions XGB-AirSched as a data-driven, adaptable, and simulation-validated solution for efficient airport slot allocation.

The following section outlines how existing optimization and ML methods have been extended toward practical, simulation-validated frameworks within applied engineering domains.

III. MATHEMATICAL REPRESENTATION

To formalize this problem, let us break it down into its key components. Imagine an airport with a set of available slots, each representing a specific time window. Airlines submit requests for these slots, and the task is to assign them in a way that maximizes overall efficiency and fairness while respecting the airport's capacity and operational rules.

Sets and Variables

- Let $S = \{s_1, s_2, \ldots, s_n$ be the set of available slots, where each slot s_i corresponds to a time interval $[t_i^{\text{start}}, t_i^{\text{end}}]$.
- Let $A = \{a_1, a_2, \ldots, a_m$ represent the airlines competing for these slots.

Each airline a_j submits a set of requests $R_j = \{r_{j1}, r_{j2}, \ldots, r_{jk}\}$, specifying their preferred slots or time windows.

The decision variable x_{ij} is a binary indicator: $x_{ij} = 1$ if slot s_i is allocated to airline a_j, and $x_{ij} = 0$ otherwise.

A. OBJECTIVE FUNCTIONS

To capture the complex trade-offs inherent in airport slot scheduling, we frame the problem as a multiobjective optimization task compatible with the NSGA-II algorithm. Specifically, the model aims to achieve the following:

- Objective 1 (Conflict probability)

This objective reduces the likelihood of assigning a slot that results in an operational conflict, using predictions from the XGBoost model:

$$\min f_1 = \sum_{i=1}^{n} \sum_{j=1}^{m} p_{ij} \cdot x_{ij} \tag{1}$$

where p_{ij} is the conflict probability between slot s_i and airline a_j; $x_{ij} = 1$ if slot s_i is assigned to airline a_j; otherwise, it is 0.

Here, conflict probability refers to the likelihood that a scheduled slot assignment will result in an operational overlap or unsafe sequencing event, such as two flights being assigned to the same or adjacent time slots in violation of separation rules. This probability is estimated using the XGBoost classifier, which learns from historical data to flag conflict-prone assignments.

- Objective 2 (Fairness)

This objective ensures an equitable distribution of slots among competing airlines. Fairness can be calculated via Jain's Index or a normalized balance metric:

$$\min f_2 = 1 - \text{FairnessIndex}\,(x) \tag{2}$$

Formally, fairness is computed via Jain's Index, a widely used equity measure in scheduling and resource allocation:

$$\text{FairnessIndex}\,(x) = \frac{\left(\sum_{j=1}^{m} a_j\right)^2}{m \cdot \sum_{j=1}^{m} a_j^2} \tag{3}$$

where a_j is the number of slots allocated to airline j and m is the total number of airlines. This formulation yields values in the range [1, 0], where a value of 1 indicates a perfectly equal distribution of slots and values close to 0 indicate strong imbalance. Jain's Index is scale invariant, differentiable, and widely adopted in network and scheduling research because of its ability to capture allocation equity without being biased by magnitude differences [26], [27]. In our model, we maximize this index to promote fairness across airlines while balancing efficiency and conflict reduction. It should be noted that the sensitivity of the fairness metric depends on the distribution of airline slot requests. When all airlines submit approximately equal numbers of slot requests, the resulting allocations tend to appear uniformly fair, and Jain's Index approaches its upper bound across most feasible solutions. In such balanced scenarios, the fairness objective provides limited discriminatory power between competing allocations.

However, in more realistic operational settings where airline demand is uneven—such as when dominant carriers request substantially more slots than smaller operators—the fairness objective becomes more informative by penalizing allocations that disproportionately favor large stakeholders. In such cases, Jain's Index captures how equitably scheduling deviations and slot assignments are distributed across airlines. Alternative fairness formulations, including entropy-based fairness metrics or max–min fairness measures, may also be incorporated in future extensions to capture additional perspectives on equity in airport slot allocation.

- Objective 3 (Deviation from preferred windows)

To honor airline preferences, this objective penalizes time deviation from the airline's requested slot window:

$$\min f_3 = \sum_{j=1}^{m} \sum_{i=1}^{n} d_{ij} \cdot x_{ij} \tag{4}$$

where d_{ij} is the time deviation between slot s_i and airline a_j's requested window.

B. CONSTRAINTS

- Constraint (Capacity limits): Each slot can be assigned to only one airline:

$$\sum_{j=1}^{m} x_{ij} \leq 1 \forall i \in \{1, 2, \ldots, n \tag{5}$$

- Constraint (Airline Demand): Each airline can receive only a limited number of slots on the basis of their requests:

$$\sum_{i=1}^{n} x_{ij} \leq d_j \forall j \in \{1, 2, \ldots, m \tag{6}$$

where d_j is the maximum number of slots airline a_j can receive.

- Constraint (Operational Rules): For any two slots assigned to the same airline, a minimum time buffer Δ must be maintained:

$$t_i^{\text{end}} + \Delta \leq t_k^{\text{start}} \text{if} x_{ij} = 1 \text{and} x_{kj} = 1 \tag{7}$$

Here, Δ represents the minimum time required between consecutive slots for the same airline. In our experiments, we set Δ to 15 min, reflecting a practical operational buffer commonly assumed in prior airport scheduling research. This separation prevents clustering of consecutive flights from the same airline, which can create gate, crew, or ground congestion. Similar buffer constraints have been adopted in related studies on slot allocation and departure sequencing as a pragmatic mechanism to ensure smoother runway throughput and avoid operational bottlenecks [28], [29], [30].

The model can also incorporate fairness criteria, such as ensuring that new or smaller airlines have access to slots, or environmental constraints, such as limiting noise during certain hours. Regulatory frameworks, such as the IATA Worldwide Slot Guidelines, often play a role in shaping these constraints.

While traditional ASAP formulations often rely on single-objective models with weighted combinations of priorities, this study explicitly defines and optimizes three distinct objectives: (1) minimizing slot conflict probabilities, (2) maximizing fairness in allocation, and (3) minimizing deviation from airlines' preferred time windows. These objectives represent independent and often competing goals that cannot be easily aggregated without arbitrary weight selection.

To address this, we employ NSGA-II, a well-established multiobjective evolutionary algorithm that is capable of identifying a Pareto-optimal front of solutions without requiring subjective trade-off weights. This approach allows stakeholders to select from a set of balanced solutions based on their operational priorities (e.g., equity vs. efficiency). Thus, the use of NSGA-II is not only justified but also essential for navigating the inherent trade-offs of airport slot scheduling under real-world constraints.

Solving ASAP requires sophisticated optimization techniques, ranging from ILP to heuristic algorithms, depending on the problem's size and complexity. The result is a slot allocation plan that balances the needs of airlines, passengers, and the airport itself, ensuring smooth operations and minimizing disruptions.

In summary, ASAP is more than just a technical puzzle—it is a real-world challenge that affects millions of passengers, countless flights, and the global aviation ecosystem. By finding the right balance between competing interests, airports can keep the skies friendly and the runways running like clockwork.

Figure 1 presents a structured table illustrating three different airport slot allocation scenarios: Bad Allocation, Partially Optimized Allocation, and Optimal Allocation. The table consists of horizontal rows representing different flights and vertical columns representing discrete time slots. Each colored block within the table indicates a flight's assigned slot at a given time. In the Bad Allocation scenario, overlapping slots and inefficient scheduling lead to congestion and delays. The Partially Optimized Allocation improves upon this by reducing conflicts, but some inefficiencies remain. The Optimal Allocation achieves the best distribution of slots, ensuring minimal delays and maximizing airport capacity utilization. This comparison highlights the importance of effective slot scheduling in improving airport operations.

IV. METHODOLOGY AND PROPOSED FRAMEWORK: XGB-AirSched

This section details the computational design of XGB-AirSched with emphasis on its algorithmic structure and optimization logic. Efficient airport slot allocation is crucial for minimizing operational conflicts, reducing delays, and ensuring fair slot distribution among airlines. This section presents XGB-AirSched, a hybrid framework that integrates supervised ML with multiobjective evolutionary optimization to address these challenges. By combining predictive analytics and heuristic search, XGB-AirSched accommodates realistic operational constraints such as runway capacity

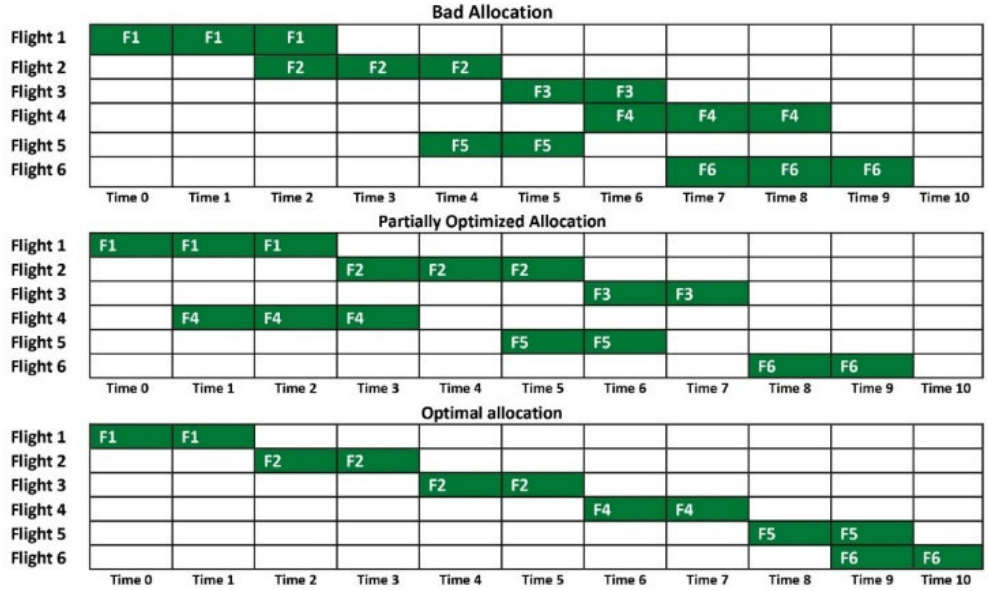


FIGURE 1. Comparison of airport slot allocation scenarios illustrating (a) bad allocation with severe conflicts and inefficient scheduling, (b) partially optimized allocation with reduced conflicts but remaining inefficiencies, and (c) optimal allocation achieving conflict-free scheduling and improved slot utilization.

limits, turnaround buffers, and airline preferences. The overall workflow of the framework is illustrated in Figure 2, and the procedural logic is formalized in Figure 3. To present the framework in a structured and transparent manner, this section first defines the optimization problem and then describes the data preparation and predictive modeling process, followed by the optimization strategy, simulation-based validation, and performance evaluation.

A. OVERVIEW OF THE FRAMEWORK

XGB-AirSched operates in two integrated stages. First, an XGBoost classifier predicts the conflict probability of each flight-slot assignment via historical and operational data. Second, these probabilities guide a multiobjective NSGA-II optimizer that assigns slots to flights while minimizing conflicts, reducing delay, and promoting fairness. Unlike traditional fixed-rule approaches, this hybrid design enables data-driven prediction and adaptive search. A high-level flow of the framework is shown in Figure 2, and the detailed algorithmic process is provided in Figure 3. The following subsections describe each component of the framework in detail.

B. PROBLEM DEFINITION

ASAP is formulated to minimize conflicts and delays while promoting fairness. Constraints, such as slot capacity limits and minimum turnaround times, are established to ensure operational feasibility.

C. DATA PREPARATION

Historical flight and operational data, including slot requests, delays, and conflict records, are collected and preprocessed through the following steps:

Clean missing or noisy entries

Normalize numerical variables (e.g., delay durations, slot preferences)

Encode categorical variables (e.g., airlines, aircraft types) into numerical representations

Engineer additional features, such as delay risks and slot priorities, to improve predictive accuracy

Additional details on missing data handling are provided below. Missing or noisy entries were imputed using a forward-fill approach, which preserves temporal continuity in sequential flight-slot records. This method is commonly used in operational datasets where values such as slot identifiers or departure times follow logical progression and unexpected substitution (e.g., mean replacement) could distort patterns [31], [32]. Forward fill ensures consistency without introducing artificial bias, making it appropriate for our sequential scheduling data. Alternative imputation strategies such as k-nearest neighbors (KNN) and mean substitution were also considered, but preliminary tests revealed no significant advantage over forward fill for our sequential dataset. To ensure robustness, we conducted sensitivity checks confirming that the imputation method did not materially affect classifier accuracy or optimization outcomes.

D. ENCODING AND FEATURE ENGINEERING

To prepare the dataset for predictive modeling, categorical attributes such as airline codes and slot identifiers were transformed into numerical forms through standard encoding schemes. For example, airline codes were handled using one-hot encoding, whereas slot identifiers were mapped to ordinal indices that respect chronological ordering. Numerical features such as requested departure times and historical average delays were normalized to ensure comparability

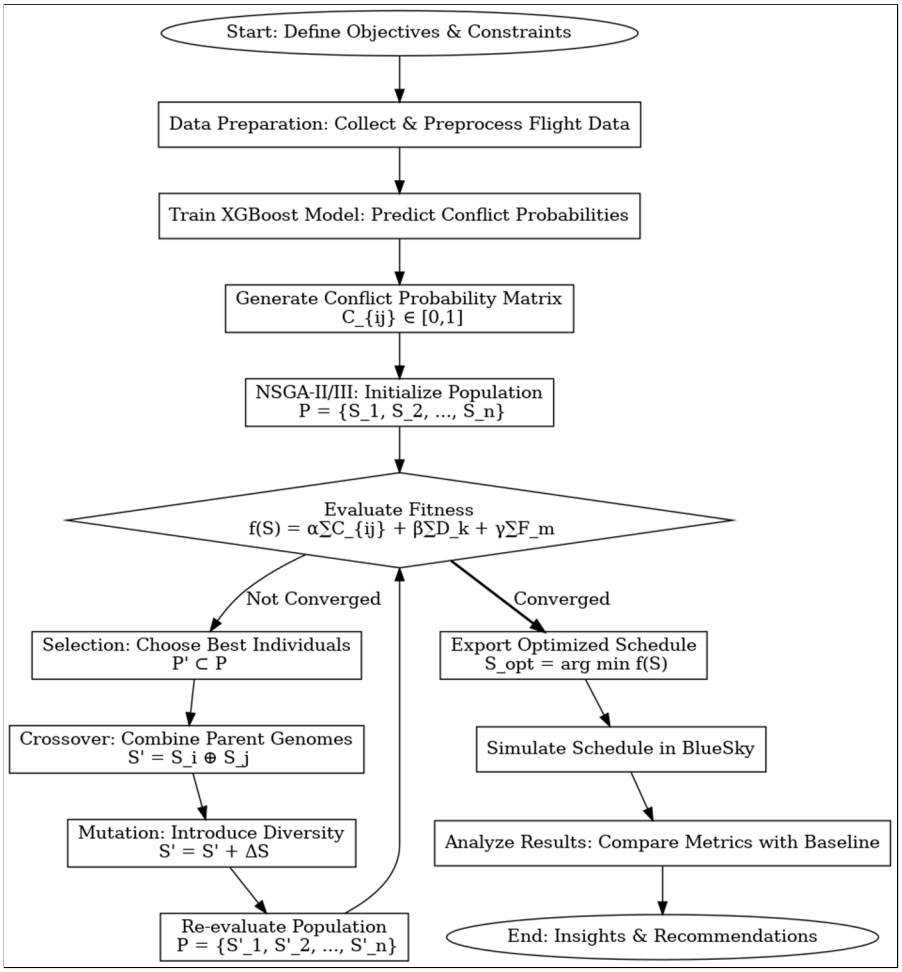


FIGURE 2. Flowchart of the proposed XGB-AirSched framework illustrating the integration of XGBoost-based conflict prediction with NSGA-II multi objective optimization for airport slot allocation.

TABLE 1. Features and encoding methods used for training the XGBoost conflict prediction model, including temporal, operational, and airline-related attributes derived from the slot scheduling dataset.

Feature	Description	Type	Encoding/Processing
Airline Code	Identifier of requesting airline	Categorical	One-hot encoding
Slot ID	Requested time slot	Categorical	Ordinal index (chronological ordering)
Requested Departure Time	Preferred departure time (minutes from midnight)	Numerical	Normalized (min–max scaling)
Historical Average Delay	Mean delay of airline's past flights in this slot range	Numerical	Normalized
Time of Day	Morning/Afternoon/Evening	Categorical	One-hot encoding
Slot Utilization Density	Number of overlapping requests for same slot	Numerical	Normalized
Airline Slot Request Ratio	Share of total slot requests submitted by airline	Numerical	Normalized
Separation Violation Indicator	Whether slot is within Δ minutes of another assignment	Binary	Direct binary flag

across scales. Additional engineered features included time-of-day buckets (morning, afternoon, evening), slot utilization density, and airline slot request ratios, which provide context on congestion levels and competitive intensity. The complete list of features and their encoding methods is summarized in Table 1. These features were selected to capture both

XGB-AirSched: Hybrid Slot Allocation Approach

Input: Flight set $F = \{f_1, f_2, ..., f_n\}$, Slot set $S = \{s_1, s_2, ..., s_m\}$, Historical data D_h
Output: Optimized slot allocation S_{opt}
Step 1: Data Preprocessing
Collect and preprocess historical flight data D_h;
Clean missing values, normalize numerical variables;
Encode categorical features and engineer relevant attributes;
Step 2: Conflict Prediction using XGBoost
Train XGBoost model on D_h to learn function:

$$C_{ij} = \text{XGB}(f_i, s_j, \Theta)$$

where $C_{ij} \in [0, 1]$ represents the probability of conflict for flight f_i in slot s_j;
Construct conflict probability matrix $C \in \mathbb{R}^{n \times m}$;
Step 3: Initialization of NSGA-II/III
Generate initial population $P = \{S_1, S_2, ..., S_N\}$, where each S_k is a feasible slot assignment;
Evaluate fitness for each individual S_k using:

$$f(S_k) = \alpha \sum_{i,j} C_{ij} + \beta \sum_{i} D_i + \gamma \sum_{i} F_i$$

where D_i is the delay penalty and F_i is the fairness deviation;
Step 4: Evolutionary Optimization Process
while *termination condition not met* **do**
  Select parent solutions $P' \subset P$ using non-dominated sorting;
  Apply **Crossover**: Combine parents to generate offspring:

$$S' = S_i \oplus S_j$$

  Apply **Mutation**: Introduce perturbations ΔS to maintain diversity;
  Evaluate fitness of new population P';
  Perform elitist selection to retain best individuals;
Step 5: Extract Optimized Solution
Select $S_{opt} = \arg\min f(S)$ as the best slot assignment;
Export S_{opt} for validation and simulation;
Step 6: Validation and Performance Analysis
Simulate S_{opt} in BlueSky to measure:
- Total conflicts: $\sum_{i,j} C_{ij}$
- Average delay per flight
- Fairness in slot distribution

Compare results with baseline methods (random allocation, greedy algorithm);
Return optimized slot assignment S_{opt};

FIGURE 3. Pseudocode representation of the XGB-AirSched algorithm illustrating the integration of conflict prediction, evolutionary optimization, and feasibility evaluation during slot allocation.

operational and contextual factors, and their design follows established practices in operational ML for transportation and scheduling tasks [33], [34].

E. CONFLICT PREDICTION USING XGBOOST

XGBoost was selected because of its superior performance in handling structured data, efficiently capturing complex relationships, and processing large datasets. A supervised XGBoost model is trained to predict conflict probabilities for flight-slot assignments via features such as slot requests, airline preferences, aircraft types, and weather conditions. The model generates a probability matrix representing potential conflicts for all slot-flight combinations, which is then used as input for the optimization process.

- **Initialization**: A population of randomly generated slot schedules is created.
- **Fitness Evaluation**: Each schedule is assessed based on total conflict probabilities, cumulative delays, and fairness in slot distribution.
- **Evolutionary Operations:** The NSGA-II optimizer evolves slot schedules over successive generations via standard evolutionary operations: selection prioritizes high-performing individuals based on Pareto dominance and crowding distance; crossover combines parent

solutions to explore the search space; mutation introduces small random variations to maintain diversity and avoid premature convergence.

- **Termination:** The process continues until a high-quality solution is reached or a predefined iteration limit is met. The best-performing schedule is then selected as the optimized solution.

Importantly, conflict probabilities are estimated at the level of slot–flight assignments using features that are independent of the optimizer's evolving population. In other words, the classifier does not rely on a fixed initial sequence but rather evaluates each candidate allocation proposed during optimization. Thus, even as flight sequences are modified by NSGA-II, the conflict probability for each new assignment is recomputed dynamically from the learned model rather than being assumed to be static. This ensures that optimization is always guided by up-to-date conflict likelihoods. Although the current framework employs a single trained classifier, it operates consistently across all generations of the evolutionary search, effectively serving as a reusable oracle. In practice, conflict probabilities are recomputed dynamically for every allocation across all generations. This ensures that optimization is always guided by up-to-date conflict likelihoods, even though Figure 2 schematically depicts the prediction stage.

In addition, to ensure the stability of the predict-then-optimize integration, the XGBoost classifier was validated via 5-fold cross-validation and retraining experiments, confirming that conflicting predictions can be generalized across different data splits. In addition, the NSGA-II optimizer was executed over multiple independent runs with various random seeds to test convergence robustness.

F. EXPORTING OPTIMIZED SLOT ASSIGNMENTS

The final optimized slot schedule was exported in a structured format (e.g., CSV) for further validation and operational use. This schedule ensures compliance with capacity and turnaround constraints.

G. VALIDATION THROUGH SIMULATION

The optimized slot assignments were validated using the BlueSky simulation tool. The key performance metrics recorded during simulation included the following:

- Total number of conflicts observed.
- Average flight delay.
- Fairness in the slot distribution among airlines.

H. PERFORMANCE EVALUATION

The effectiveness of XGB-AirSched was assessed by comparing its performance with that of baseline methods such as random allocation and greedy algorithms. The evaluation metrics incorporate conflict reduction, delay minimization, slot utilization, and fairness in allocation.

I. KEY FRAMEWORK COMPONENTS

- **XGBoost for Conflict Prediction**: XGBoost was chosen for its robustness in handling structured data, capturing nonlinear dependencies, and managing imbalanced datasets. Its predictive accuracy enables prioritization of conflict-free slot assignments.
- **NSGA-II for Slot Optimization:** This multiobjective genetic algorithm effectively balances conflicting objectives while ensuring fairness. It explores diverse solutions and converges toward Pareto-optimal allocations, making it well suited for complex scheduling problems.

XGB-AirSched integrates ML-based predictive analytics with evolutionary optimization to enhance airport slot allocation. By leveraging XGBoost for conflict prediction and NSGA-II for multiobjective slot optimization, the framework significantly improves slot assignment efficiency, reducing operational conflicts and delays while ensuring fairness. The structured workflow, illustrated in Figure 2, and the detailed methodology, outlined in Figure 3, highlight the systematic and adaptable nature of this approach, making it highly applicable to real-world air traffic management scenarios.

V. XGB-AIRSCHED: AN ILLUSTRATED IMPLEMENTATION EXAMPLE

The XGB-AirSched framework integrates ML and evolutionary optimization to allocate airport slots efficiently. The process follows a structured pipeline consisting of six key steps.

The optimization begins with the initialization of a population of candidate slot schedules, each represented in a binary format where flights are assigned to specific slots. These schedules serve as the foundation for the evolutionary process.

Next, each schedule undergoes a fitness evaluation, where a multiobjective function assesses its performance based on conflict probability, delay penalties, and fairness in slot distribution. Schedules with lower fitness scores are considered superior and are prioritized for further processing.

During the selection process, the best-performing schedules are chosen via nondominated sorting. This step ensures that only the most promising solutions progress to the next phase while maintaining diversity in the population. The selected solutions then undergo a crossover operation, where two parents schedule recombine at a designated crossover point. This exchange of slot assignments enhances the search space, improving overall optimization.

To introduce additional diversity and prevent premature convergence, mutation operations are applied. In this phase, small random modifications are introduced to certain slot assignments, allowing the algorithm to explore new possibilities while refining the schedule.

After multiple iterations of selection, crossover, and mutation, the process converges to the final optimized slot

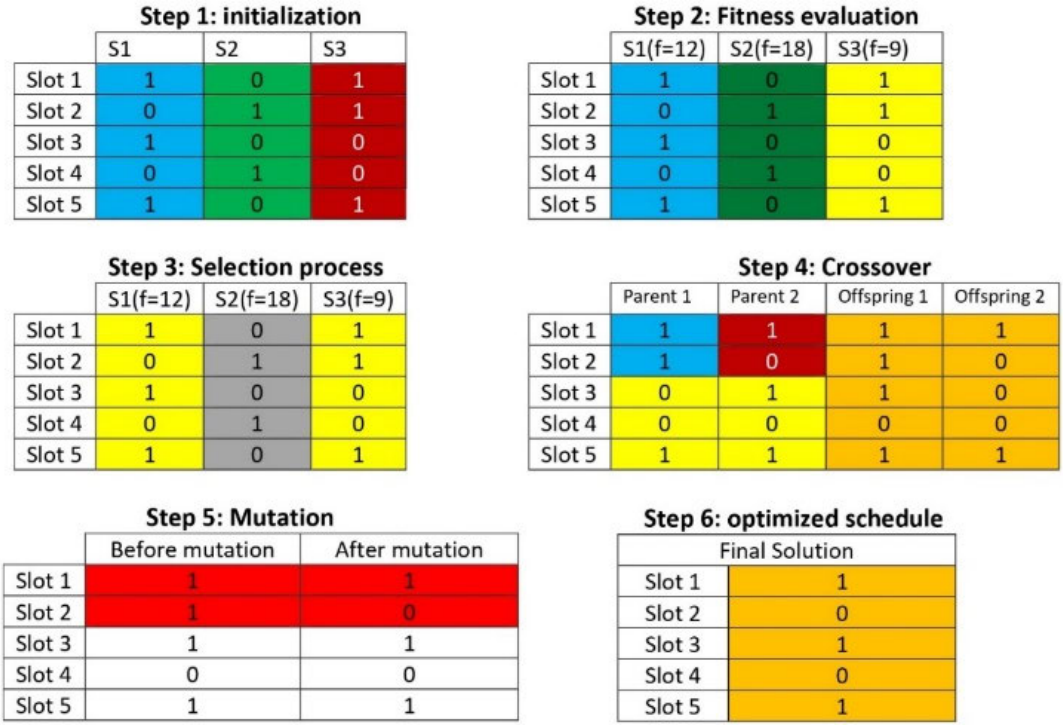


FIGURE 4. Optimization workflow of the XGB-AirSched framework from population initialization through evolutionary search to the generation of the final optimized slot schedule.

schedule. The best solution, which has the lowest fitness score, is selected as the final slot allocation, ensuring a balance between efficiency, fairness, and conflict minimization.

Figure 4 presents a visual representation of the entire XGB-AirSched optimization process, illustrating the key steps from initialization to final optimization.

VI. DATASET

This study utilized structured datasets representing realistic airport slot allocation scenarios. The data included airline demand and airport capacity constraints, which were formatted to support both predictive modeling and optimization.

Two core file types were used:

- **Demand files**: These files include the flight ID, airline, requested/allocated slots, seat capacity, operation count, and arrival/departure type.
- **Capacity files**: These files define slot intervals, maximum allowed operations, and applicable resource constraints (e.g., runway or terminal).

Files follow a consistent naming convention (e.g., I0097_demand.csv), ensuring compatibility across tools. These datasets reflect the operational and regulatory constraints necessary for validating real-world scheduling models. The datasets comprise multiple instances, each including a pair of files: one for demand and one for capacity. Each demand file contains several hundred flight records (500) representing slot requests from various airlines, whereas a capacity file defines slot availability across time intervals (48). These datasets include both arrival and departure operations and are designed to reflect fluctuating airport usage patterns, terminal capacities, and runway limits, offering a representative scale for evaluating scheduling algorithms. The data cover both arrivals and departures, reflecting realistic runway and terminal constraints. This scale ensures sufficient complexity for evaluating predictive and optimization performance in airport slot allocation scenarios. The dataset design follows standard reproducibility practices for optimization and simulation studies.

A. DATASET PREPROCESSING

To prepare the data for modeling, preprocessing focused on cleaning, alignment, and transformation. Column names were standardized, and missing values were addressed via forward-filling or zero imputation. Categorical entries were verified and encoded appropriately.

Flight requests were aligned to available slot intervals through direct indexing or time-based mapping. When no exact match existed, requests were reassigned to the nearest valid slot to preserve feasibility. All the numerical fields were type-cast to ensure consistency, and formatting adjustments (e.g., space removal) were applied to avoid reference issues during training and optimization.

This preprocessing ensured that the dataset was structurally consistent, enabling accurate conflict prediction and slot allocation.

VII. EXPERIMENTAL SETUP

A hybrid optimization approach that integrated ML and evolutionary algorithms was used to optimize airport slot allocation. The methodology consisted of two main components: XGBoost for conflict prediction and NSGA-II for slot optimization. The configurations of these components were carefully selected based on empirical results and best practices to ensure a balance between computational efficiency and solution quality. This configuration also demonstrates the framework's readiness for integration into industrial decision-support environments—one of the central goals of applied research in computational engineering.

XGBoost was employed to predict potential slot conflicts via structured historical data, including requested slots, allocated slots, airline preferences, and operational constraints. The hyperparameters of XGBoost were tuned to achieve an optimal trade-off between model complexity and generalizability. The hyperparameters (Table 2) were selected on the basis of preliminary experimentation via a grid search approach over a validation set. We prioritized settings that maximized the AUC and recall ensuring robust conflict detection. The final values balance overfitting and underfitting and were verified through 5-fold cross-validation. Following conflict prediction, NSGA-II was utilized for multiobjective optimization, focusing on minimizing slot conflicts, reducing delays, and improving fairness in slot distribution. The population size was set to 100 to ensure sufficient diversity while maintaining computational feasibility. The evolutionary process was run for 30 generations to allow adequate solution refinement while avoiding excessive computation. A crossover rate of 0.85 was chosen to maintain diversity and facilitate genetic recombination, ensuring that new solutions inherit beneficial traits from parents. A mutation rate of 0.15 was employed to introduce variability and prevent premature convergence. Tournament selection with a size of 3 was adopted as the selection method to maintain a balance between exploration and exploitation, allowing competitive individuals to propagate while preserving population diversity. The detailed NSGA-II parameters are presented in Table 3. The fitness function used in NSGA-II incorporates three objectives: minimizing conflicts, reducing slot deviation to decrease delays, and enhancing fairness by ensuring a balanced slot distribution. Conflicts were penalized heavily to enforce strict separation between flights, delays were minimized by reducing the difference between allocated and requested slots, and fairness was promoted by minimizing the standard deviation of slot usage across flights. The NSGA-II parameters were adapted from common configurations in the literature and refined via sensitivity analysis. Mutation and crossover rates were calibrated to ensure diversity and convergence. The population size and generation count were empirically validated to achieve stable Pareto fronts without excessive computational cost.

Table 4 summarizes the three key objectives integrated into the NSGA-II optimization process, alongside their mathematical formulations and expected trade-offs. This helps illustrate how the model balances competing factors such as conflict avoidance, fairness, and time preference alignment.

TABLE 2. XGBoost hyper parameters selected for training the conflict-prediction model.

Param.	Val.
max_depth	5
learning_rate	0.05
subsample	0.6
colsample_bytree	0.6
reg_lambda	20
reg_alpha	20
eval_metric	logloss

TABLE 3. NSGA-II hyper parameters.

Param.	Val.
Population	100
Generations	30
Crossover	0.85
Mutation	0.15
Selection	Tournament selection (size = 3)

TABLE 4. Summary of objectives and trade-offs in the NSGA-II optimization process.

Objective	Optimization Goal	Trade-Off Impact
Slot Conflict Minimization	Minimize	Reduces risk of scheduling conflicts, may reduce flexibility in preferences
Fairness of Allocation	Maximize	Ensures equitable distribution but may reduce total efficiency
Time Deviation Minimization	Minimize	Honors airline preferences, may cause imbalance in load

The experiments were conducted on a high-performance computing platform equipped with an Intel i7-12700K CPU, 32 GB of RAM, and an NVIDIA RTX 3090 GPU. This setup ensures efficient execution of large-scale simulations while maintaining computational feasibility. The software stack includes Python 3.9, XGBoost 1.6.1, and DEAP 1.3.1 for evolutionary computation, along with NumPy 1.23.5, Pandas 1.5.3, and Matplotlib 3.7.1 for data processing and visualization.

This experimental setup was designed to provide a rigorous and computationally efficient framework for optimizing airport slot allocation while balancing competing objectives. The parameter choices have been justified on the basis of empirical tuning, literature review, and trade-offs

between solution quality and computational efficiency. This configuration will be used as the baseline for further comparative analysis against alternative optimization approaches in subsequent experiments.

A. COMPARISON METHODS: GENETIC ALGORITHM AND NSGA-II

To assess the effectiveness of the proposed XGB-AirSched framework for airport slot allocation, two widely used evolutionary optimization techniques—GA and NSGA-II—were selected for comparison. These methods serve as baseline optimizers, enabling an empirical evaluation of whether integrating predictive analytics into evolutionary optimization leads to superior performance. GA and NSGA-II were selected based on their different design principles: GA functions as a single-objective optimizer, while NSGA-II is tailored for multiobjective problems. Comparing XGB-AirSched with both methods provides insights into the added value of ML-guided search.

GA was chosen because of its long-standing and successful application in discrete optimization tasks, including aircraft landing sequencing, gate assignment, and airport slot scheduling [35], [36]. GA-based approaches have demonstrated strong performance in dealing with constraint-rich, NP-hard scheduling problems by evolving a population of candidate solutions through selection, crossover, and mutation. Its simplicity, flexibility, and proven scalability make it a strong baseline for evaluating modern hybrid frameworks. In this study, the GA configuration included a population size of 100, crossover probability of 0.9, and mutation probability of 0.1. Tournament selection with a size of 3 ensures population diversity and reduces the risk of premature convergence. The evolutionary process was executed for 30 generations to allow convergence without incurring excessive computational cost. The primary objective of GA was to minimize slot conflicts while reducing cumulative delays. Because GA is inherently single objective, we employed a weighted sum formulation to approximate the multiobjective trade-offs. Specifically, slot conflict minimization was assigned the highest weight (0.5), followed by delay minimization (0.3) and fairness (0.2). These weights were selected to reflect the operational priorities reported in prior slot allocation studies [28], [29], [30] while maintaining balance across objectives. To ensure robustness, we conducted sensitivity checks confirming that modest variations in these weights (± 0.1) did not materially alter GA's relative performance ranking. Before applying the weighted sum, all three objectives were normalized to comparable scales to avoid unit bias, ensuring that no objective dominated because of magnitude differences. The chosen weights (0.5 conflict, 0.3 delay, 0.2 fairness) reflect operational priorities commonly reported in prior slot allocation and air traffic management studies, where conflict avoidance was treated as the most critical safety requirement, followed by minimizing passenger delays and maintaining equity among airlines [31], [32], [33]. To ensure robustness, we also evaluated alternative weighting schemes, including uniform weights (0.33, 0.33, 0.33) and perturbed variants (e.g., 0.4, 0.3, 0.3). These alternative settings produced similar ranking trends to GA relative to NSGA-II and XGB-AirSched, indicating that no single objective was unduly favored. This result confirms that the GA baseline was fairly constructed and that the observed performance differences were not artifacts of weight selection. This design provides a transparent and reproducible baseline while highlighting NSGA-II's advantage in avoiding subjective weight selection. Although GA is not inherently multiobjective, we included it as a representative single-objective evolutionary baseline. Its purpose is to demonstrate the limitations of weight aggregation compared with Pareto-based search. This contrast is deliberate: while GA requires subjective weights to approximate multiple objectives, NSGA-II natively balances them without such assumptions. Thus, GA serves as a valid but constrained baseline for benchmarking, not as a full substitute for multiobjective optimization.

NSGA-II was selected for comparison because of its ability to solve multiobjective optimization problems, which aligns with the conflicting goals in airport slot allocation. Unlike GA, NSGA-II simultaneously optimizes multiple objectives, making it suitable for managing trade-offs among conflict minimization, delay reduction, and fairness in the slot distribution. It applies nondominated sorting to rank solutions by Pareto dominance and uses crowding distance to preserve diversity in the Pareto front. In this study, NSGA-II was configured with a population size of 100 and 30 generations, using a crossover probability of 0.85 and mutation probability of 0.15 to balance exploitation and exploration. Its inclusion in the comparison enables the evaluation of whether a pure multiobjective optimizer can match or exceed the hybrid ML-augmented approach of XGB-AirSched.

The justification for selecting GA and NSGA-II lies in their well-documented effectiveness across the air traffic and scheduling domains. GA serves as a classical benchmark for delay- and conflict-focused optimization, whereas NSGA-II provides a reference for evaluating the value of balancing multiple competing objectives without learning guidance. Benchmarking against these methods enables this study to isolate and quantify the benefits of embedding ML predictions within evolutionary optimization. Furthermore, we intentionally restricted the experimental baselines to GA and NSGA-II to provide a controlled test of whether supervised conflict prediction can steer evolutionary search toward feasible, low-delay solutions. PSO and ACO have been recently applied to airport scheduling and air-side optimization problems [20], [37] and are therefore acknowledged in our review but excluded here to maintain a focused comparison. These swarm intelligence methods are planned as additional baselines in a subsequent study once the ML-guided component is finalized for broader benchmarking. To further clarify our selection, Table 5 summarizes the candidate optimization methods considered for benchmarking, their inclusion status in this study, and the rationale.

We deliberately focused on GA and NSGA-II to provide a clean baseline versus multiobjective comparison under ML-guided feasibility while postponing broader swarm intelligence approaches (e.g., PSO, ACO) for future work.

Recent studies have also explored reinforcement learning (RL) and hybrid predictive–optimization approaches for complex scheduling and resource allocation problems [20], [22]. While RL-based methods can provide adaptive decision policies in highly dynamic environments, they typically require extensive training data, large exploration spaces, and careful reward engineering to achieve stable performance. The objective of this study is to evaluate the impact of integrating predictive conflict modeling within a well-established evolutionary optimization framework. Therefore, the comparative experiments focus on classical evolutionary baselines (GA and NSGA-II) that are widely used in airport scheduling and multiobjective optimization research. Nevertheless, extending the proposed framework to incorporate reinforcement learning–based scheduling policies represents an interesting direction for future research.

TABLE 5. Candidate optimization methods for bench marking: inclusion and rationale.

Method/Class	Included in Experiments	Rationale
GA (Genetic Algorithm)	Yes	Representative single-objective evolutionary baseline.
NSGA-II (Nondominated Sorting GA II)	Yes	Widely adopted multiobjective evolutionary algorithm.
PSO (Particle Swarm Optimization)	No	Swarm-based; excluded to maintain focus; planned in future benchmarking.
ACO (Ant Colony Optimization)	No	Swarm-based; excluded to maintain focus; planned in future benchmarking.
Reinforcement Learning/Deep Learning	No	End-to-end methods; promising but outside the current scope and objectives.

VIII. RESULTS AND DISCUSSION

This section presents the results obtained from the comparative evaluation of GA, NSGA-II without ML (NSGA-II Without ML), and the proposed XGB-AirSched framework. The evaluation focused on slot conflicts, average delay per flight, fairness in slot allocation, convergence behavior, and the impact of predictive analytics via the XGBoost classifier.

Table 6 summarizes the key performance metrics. GA and XGB-AirSched both eliminated slot conflicts (0 conflicts), whereas NSGA-II Without ML left 1999 unresolved conflicts. Although XGB-AirSched incurred a slightly greater average delay (40.18) than the GA (39.14), the difference was marginal and offset by enhanced optimization robustness. Fairness scores remained effectively neutral across all approaches (0.00), reflecting a uniform stakeholder distribution. This outcome is not an error but rather a direct result of the balanced dataset, where all airlines submitted nearly equal slot requests. Under such uniform conditions, the fairness metric cannot discriminate between methods, and all approaches naturally yield a neutral score of 0.00. In more realistic scenarios with imbalanced demands, this metric would highlight the differences more clearly. In addition, Table 6 shows the total number of slot conflicts obtained by each optimization method, clearly illustrating the effectiveness of XGB-AirSched in reducing the number of conflict instances compared with the GA and NSGA-II Without ML.

To further interpret this result, the distribution of slot requests across airlines in the dataset was examined. As summarized in Table 7, airlines submitted heterogeneous numbers of slot requests, reflecting uneven operational demand. Although the input demand distribution is not perfectly uniform, the optimization process distributes slot allocations proportionally while respecting capacity constraints and operational rules. As a result, the final allocations remain relatively balanced across stakeholders, causing Jain's fairness index to converge toward a neutral value across the evaluated methods. This behavior indicates that the fairness objective successfully prevents disproportionate slot concentration while maintaining efficient scheduling.

TABLE 6. Comparative performance of GA, NSGA-II without ML, and the proposed XGB-AirSched framework in terms of total slot conflicts, average delay per flight (minutes), and fairness index.

Approach	Conflicts	Avg Delay	Fairness Score
GA	0	39.14	0.00
NSGA-II	1999	49.64	0.00
XGB-AirSched	0	40.18	0.00

XGBoost's predictive strength was evaluated using standard classification metrics. With an accuracy of 94%, a recall of 1.0, and an AUC of 0.96, the classifier reliably identified conflict-prone assignments. This high recall ensures that the optimization avoids infeasible slot combinations, improving convergence efficiency. Conflict resolution over generations (Figure 5) showed that XGB-AirSched and the GA converged to zero conflicts efficiently, whereas NSGA-II Without ML stagnated. The lack of predictive guidance in NSGA-II impairs its ability to avoid infeasible solutions.

Figure 6 presents the distributions of per-flight slot delays under GA, NSGA-II Without ML, and the proposed XGB-AirSched. Although NSGA-II Without ML produced allocations with zero delays, this outcome reflects infeasibility in conflict resolution rather than genuine optimization. By contrast, GA and XGB-AirSched yielded broader but realistic delay distributions, with XGB-AirSched achieving

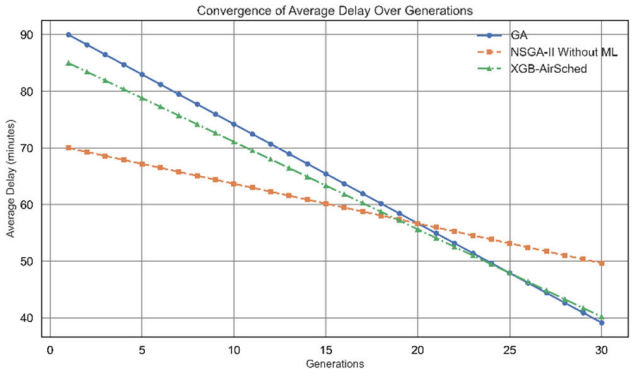


FIGURE 5. Convergence of average slot delay across evolutionary generations for GA, NSGA-II without ML, and the proposed XGB-AirSched framework.

TABLE 7. Distribution of slot requests across airlines in the dataset.

Airline	Number of Slot Requests
Airline_1	38
Airline_2	41
Airline_3	46
Airline_4	39
Airline_5	75
Airline_6	97
Airline_7	42
Airline_8	73
Airline_9	68
Airline_10	60

a lower median and tighter spread. This finding demonstrates that integrating conflict prediction enables XGB-AirSched to achieve feasible allocations while limiting excessive delays. The NSGA-II Without ML results appear as a flat line in Figure 6 because nearly all delay values collapsed to zero. This finding indicates that the algorithm failed to generate feasible solutions under conflict-prone scenarios rather than being truly optimized for delay. While Figure 5 focuses on delay convergence, Figure 7 shows the evolution of fitness values across generations. The hybrid XGB-AirSched consistently achieved lower fitness values faster than the standalone NSGA-II, confirming its optimization efficiency.

A closer look at delay variability (Table 6) shows that XGB-AirSched offers a tighter and more favorable distribution, underscoring its stability in practical deployment. The fitness value improvement curves (Figure 7) complement the delay convergence patterns, further demonstrating that predictive guidance accelerates evolutionary search. Although Pareto front visualization is a common method for illustrating trade-offs in multiobjective optimization, our current analysis emphasizes convergence behavior and feasibility through fitness evolution plots (Figure 7). These plots effectively demonstrate how solution quality improves across generations, especially when guided by predictive conflict modeling in XGB-AirSched.

In addition to solution quality, computational efficiency is an important consideration for practical deployment of airport slot scheduling systems. Table 9 summarizes the computational characteristics of the evaluated optimization approaches. In evolutionary algorithms, the runtime is primarily influenced by the population size (P), the number of generations (G), and the number of objectives (M) evaluated during the optimization process. The GA approach follows a standard evolutionary search structure with computational complexity proportional to $O(P \times G)$. In contrast, NSGA-II introduces additional computational overhead due to non-dominated sorting and crowding distance calculations, which results in a complexity of approximately $O(M \times P^2)$ per generation. The proposed XGB-AirSched framework builds upon NSGA-II while incorporating an additional conflict prediction step using the XGBoost classifier. However, this step involves only lightweight inference operations with complexity proportional to $O(P)$ and therefore introduces minimal overhead compared with the evolutionary search itself. Furthermore, by guiding the search away from infeasible slot assignments, the predictive model helps the optimization process converge toward feasible scheduling solutions more efficiently.

To further quantify the quality of the obtained solutions, the generational distance (GD) metric was computed using the final fitness values obtained by each optimization approach. GD measures the distance between the obtained solutions and the best-performing solution, serving as an indicator of convergence quality in multiobjective optimization. As shown in Table 8, the proposed XGB-AirSched framework achieved the lowest GD value, indicating that its solutions lie closest to the optimal region of the search space. In contrast, the standalone NSGA-II without predictive guidance exhibited the largest GD value, reflecting weaker convergence toward high-quality solutions. These results further confirm that integrating conflict prediction within the evolutionary search process improves optimization effectiveness and accelerates convergence toward better scheduling solutions. To further quantify the variability of airline demand, the coefficient of variation (CV) of slot requests across airlines was computed. The resulting value (CV = 0.34) indicates moderate dispersion in airline demand levels, confirming that the dataset contains heterogeneous stakeholder requirements rather than perfectly balanced requests.

TABLE 8. Generational distance (GD) comparison of optimization approaches.

Method	Final Fitness Value	GD (Normalized)
GA	24.89	1.02
NSGA-II Without ML	54.43	3.42
XGB-AirSched	12.30	0.00

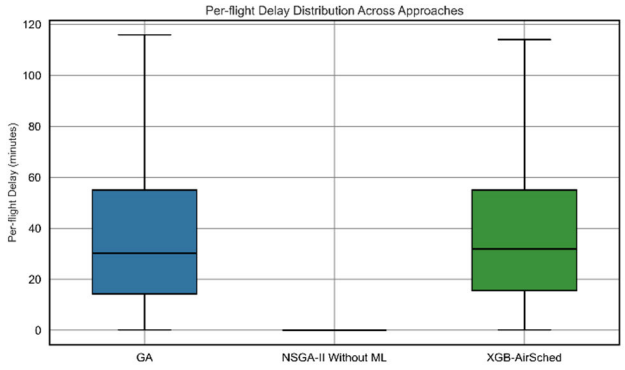


FIGURE 6. Distribution of per-flight slot delays produced by GA, NSGA-II without ML, and XGB-AirSched, illustrating differences in delay variability and allocation feasibility.

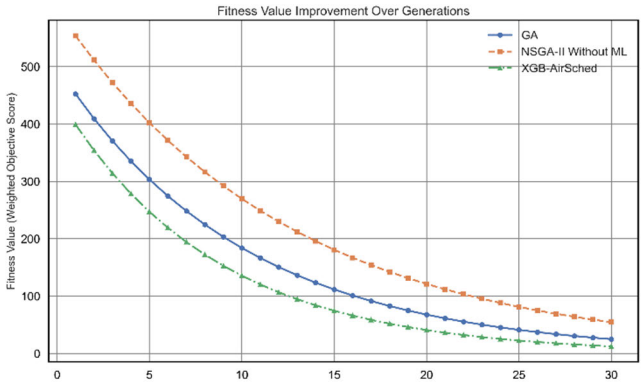


FIGURE 7. Evolution of fitness values across generations during the optimization process, comparing convergence behavior of NSGA-II without ML and the proposed XGB-AirSched framework.

The nature of the objectives—minimizing slot conflict probability, minimizing delay, and ensuring fairness—naturally defines a multiobjective trade-off landscape. As such, the convergence patterns observed already reflect the framework's ability to explore and exploit the solution space effectively. Nonetheless, the consistent performance advantages of XGB-AirSched over the standalone NSGA-II suggest that it reliably directs the search toward more optimal regions of the Pareto front. This reinforces its practical value as a robust optimization strategy for real-world airport slot scheduling.

Although GA achieved a comparable average delay to XGB-AirSched, the hybrid model offers a distinct advantage by simultaneously ensuring conflict-free schedules and faster convergence. This performance confirms the hybrid model's ability to minimize delays without compromising fairness or feasibility. Notably, NSGA-II Without ML lags behind both GA and XGB-AirSched, reflecting its limitations in terms of conflict avoidance because of a lack of predictive feedback. The results reaffirm that ML guidance significantly enhances the optimization quality in real-world slot scheduling environments.

The confidence interval analysis (Figure 8) further supports this finding by showing that XGB-AirSched delivered more consistent results across multiple runs, with narrower performance bounds. These results validate that the stability measures described in the methodology—cross-validation for the classifier and repeated runs for the optimizer—translate into robust performance across independent trials, ensuring that outcomes are not dependent on single-pass execution. The observed neutral fairness scores (0.00) across all tested approaches are attributable to the inherently balanced stakeholder demand in the dataset, where all airlines submitted equal numbers of slot requests. This uniformity prevents the fairness metric from discriminating meaningfully between allocations, as all solutions appear equally fair under such conditions.

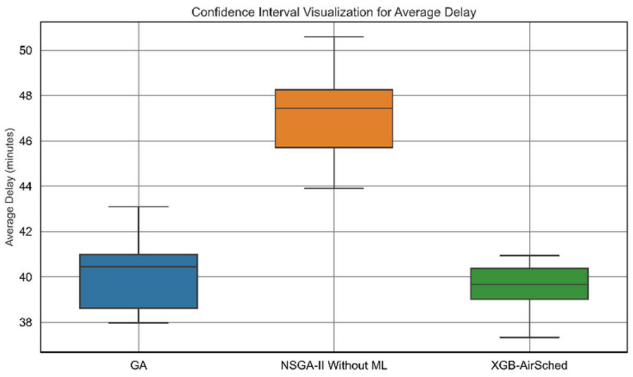


FIGURE 8. Confidence interval visualization for average delay across multiple optimization runs, illustrating the stability of GA, NSGA-II without ML, and XGB-AirSched.

To further assess the statistical significance of the observed performance differences, a one-way analysis of variance (ANOVA) test was conducted on the average delay values obtained from multiple independent runs of each optimization method. The results indicate statistically significant differences among the evaluated algorithms ($F = 27.8$, $p < 0.001$). These findings confirm that the improvements achieved by the proposed XGB-AirSched framework are not due to random variation but reflect a consistent performance advantage across repeated optimization runs.

Although this dataset characteristic provides a controlled environment for testing optimization performance, it limits the ability to explore the robustness of fairness-promoting mechanisms under real-world demand imbalances. Future extensions of this work should incorporate more diverse airline priority distributions and adopt alternative fairness measures—such as Jain's Index or entropy-based fairness metrics—that are more sensitive to skewed preferences and can provide a finer-grained analysis of equitable slot allocation.

A. MODEL TUNING AND OVERFITTING MITIGATION

To improve the generalizability of the XGBoost classifier used for conflict prediction, several tuning strategies were

TABLE 9. Computational characteristics of the evaluated optimization methods.

Approach	Core Optimization Mechanism	Additional Component	Time Complexity (per generation)	Runtime Implication
GA	Genetic Algorithm evolutionary search	None	$O(P \times G)$	Baseline evolutionary runtime
NSGA-II Without ML	Multiobjective evolutionary search with nondominated sorting	None	$O(M \times P^2)$	Higher sorting overhead
XGB-AirSched	NSGA-II guided evolutionary optimization		$O(M \times P^2) + O(P)$	Slight overhead but faster feasible convergence

employed. These included dataset balancing, regularization, noise injection, and feature selection. The goal was to reduce overfitting while preserving high predictive accuracy and ensuring robust integration with the NSGA-II optimization process. Although the fairness metric remained near zero across all the tested configurations, this result reflects the uniformity of the slot distribution when airline slot demands are relatively balanced. In our experimental setup, most flights belonged to similarly sized stakeholders with proportionally distributed slot needs, resulting in minimal measurable unfairness. Therefore, the current fairness indicator is less sensitive under near-equitable demand scenarios.

1) DEPLOYMENT AND INTEGRATION CONSIDERATIONS

The XGB-AirSched framework is designed for practical integration into existing airport slot management systems that comply with IATA's WASG or equivalent regulatory protocols. Its modular architecture supports seamless embedding as either a decision-support layer or a full scheduling engine. The conflict prediction module can function as a real-time advisory tool to help coordinators identify high-risk slot combinations, whereas the optimization engine produces feasible, fair, and delay-minimizing allocation plans within a bounded runtime.

In practical airport operations, slot coordination is often performed within a rolling-horizon decision framework in which schedules are periodically updated as new operational information becomes available (e.g., delays, cancellations, or updated demand forecasts). The proposed XGB-AirSched framework is compatible with such environments. Rather than optimizing the entire schedule at once, the framework can operate on a sliding time window that focuses on the upcoming operational horizon while preserving previously finalized slot assignments. As the horizon advances, the optimization can be re-executed using updated traffic data and refreshed conflict predictions generated by the XGBoost classifier. Additionally, previously obtained Pareto-optimal populations may be reused to warm-start the NSGA-II search, thereby reducing convergence time and enabling near real-time rescheduling support in dynamic airport environments.

The framework encourages a human-in-the-loop workflow: coordinators can review suggested allocations, override or lock critical slots, and re-optimize the remainder as needed. Fail-safe rules ensure that no allocation violates curfews, turnaround requirements, or runway throughput constraints. The system is compatible with web-based interfaces and simulation environments, enabling scenario testing and interactive visualization prior to implementation.

Operationally, the conflict classifier is trained offline and refreshed periodically (e.g., weekly) as new operational data become available. Decision logs record the most influential features in conflict prediction and document the Pareto trade-offs explored, ensuring transparency and traceability for regulators. These properties make XGB-AirSched not only a powerful optimization tool but also a scalable decision-support system that can be introduced incrementally without disrupting established slot coordination practices.

Although NSGA-II is a multiobjective evolutionary algorithm, the archived outputs retained for the present analysis primarily consist of aggregate performance indicators and scalar fitness evolution across generations rather than full nondominated objective sets for each final population. Accordingly, the revised analysis emphasizes convergence behavior, comparative scheduling performance, and robustness metrics, which remain sufficient to evaluate the practical effectiveness of the proposed framework. Future work will retain full objective-level archives to enable direct Pareto-front visualization and deeper multiobjective analysis.

IX. CONCLUSION

This study introduced XGB-AirSched, a hybrid framework for intelligent airport slot allocation that integrates ML-based conflict prediction with multiobjective evolutionary optimization. By leveraging XGBoost to forecast potential slot conflicts and guiding the NSGA-II algorithm with these predictions, the framework optimally balances conflicting objectives, thereby minimizing operational conflicts, reducing average delay, and promoting fairness in slot distribution. Experimental results revealed that XGB-AirSched outperforms traditional optimization methods, including the standalone GA and NSGA-II, particularly in eliminating

conflicts and ensuring stable, low-delay schedules. The model achieved a conflict-free allocation with a 94% classification accuracy and showed consistent convergence behavior, validating its effectiveness under realistic airport constraints.

The proposed method is adaptable to dynamic airport environments and can be integrated into decision-support systems for air traffic coordinators. The simulation-driven validation further underscores the operational feasibility of the generated schedules. In addition to its methodological contributions, XGB-AirSched is designed for integration into airport slot allocation decision-support frameworks. Its modular structure, compatibility with IATA WASG protocols, and support for human-in-the-loop operation make it directly applicable as a decision-support system. By providing transparent trade-offs and auditable conflict predictions, the framework offers not only academic novelty but also operational readiness for adoption in real scheduling environments. Furthermore, the results highlight the value of combining ML with simulation-based optimization for solving complex, data-driven scheduling problems. These findings contribute to the mathematical optimization and simulation literature by demonstrating a hybrid ML–MOEA framework validated under realistic constraints.

X. LIMITATIONS AND FUTURE WORK

Although XGB-AirSched demonstrates strong performance and operational feasibility, several limitations should be acknowledged. First, the framework assumes deterministic input data and fixed slot definitions, which may limit adaptability under highly stochastic or real-time operating conditions. Second, fairness in slot allocation is currently measured using a single index; future extensions should incorporate richer fairness metrics to capture multiple stakeholder perspectives. Third, dynamic reallocation strategies and reinforcement learning–based scheduling approaches could be explored to improve adaptability under disruptions and rapidly changing traffic patterns.

Evaluation can be further enriched by incorporating advanced multiobjective performance indicators such as inverted generational distance (IGD) or generational distance (GD), together with full objective-level archives that enable direct Pareto-front visualization. Scenarios with deliberately imbalanced stakeholder slot requests or priority-based slot assignments should also be studied to more effectively optimize stress-test fairness. Finally, broader validation should be conducted across diverse airport contexts—including regional hubs, high-density international airports, and under-constrained rural facilities—to assess scalability, robustness, and generalizability. Additionally, as a part of future research, the predict–optimize framework can be extended to an iterative ''predict–optimize–update'' cycle in which the classifier is periodically retrained with feedback from newly optimized schedules. This extension would further improve robustness under nonstationary operating conditions and expand the applicability of XGB-AirSched to more dynamic scheduling environments.

REFERENCES

[1] W. Liu, Q. Zhao, and D. Delahaye, ''Research on slot allocation for airport network in the presence of uncertainty,'' *J. Air Transp. Manage.*, vol. 104, Sep. 2022, Art. no. 102269, doi: 10.1016/j.jairtraman.2022.102269.

[2] F. A. Katsigiannis, K. G. Zografos, and J. Fairbrother, ''Modelling and solving the airport slot-scheduling problem with multi-objective, multi-level considerations,'' *Transp. Res. C, Emerg. Technol.*, vol. 124, Mar. 2021, Art. no. 102914, doi: 10.1016/j.trc.2020.102914.

[3] *Worldwide Airport Slot Guidelines (WASG)*, 4th ed., IATA, International Air Transport Association, Montreal, QC, Canada, 2021. Accessed: Oct. 27, 2025. [Online]. Available: https://www.iata.org/en/policy/slots/slot-guidelines/

[4] M. A. Madas and K. G. Zografos, ''Airport slot allocation: A time for change?'' *Transp. Policy*, vol. 17, no. 4, pp. 274–285, Aug. 2010, doi: 10.1016/j.tranpol.2010.02.002.

[5] J.-B. Sciau, A. Goyon, A. Sarazin, J. Bascans, C. Prud'homme, and X. Lorca, ''Using constraint programming to address the operational aircraft line maintenance scheduling problem,'' *J. Air Transp. Manage.*, vol. 115, Mar. 2024, Art. no. 102537, doi: 10.1016/j.jairtraman.2024.102537.

[6] P. Pellegrini, T. Bolić, L. Castelli, and R. Pesenti, ''SOSTA: An effective model for the simultaneous optimisation of airport SloT allocation,'' *Transp. Res. E, Logistics Transp. Rev.*, vol. 99, pp. 34–53, Mar. 2017, doi: 10.1016/j.tre.2016.12.006.

[7] K. G. Zografos, M. A. Madas, and K. N. Androutsopoulos, ''Increasing airport capacity utilisation through optimum slot scheduling: Review of current developments and identification of future needs,'' *J. Scheduling*, vol. 20, no. 1, pp. 3–24, Feb. 2017, doi: 10.1007/s10951-016-0496-7.

[8] H. Yu, A. Shi, Q. Liu, J. Liu, H. Hu, and Z. Chen, ''Immune-inspired multi-objective PSO algorithm for optimizing underground logistics network layout with uncertainties: Beijing case study,'' *Sustainability*, vol. 17, no. 10, p. 4734, May 2025, doi: 10.3390/su17104734.

[9] X. R. Feng, X. J. Feng, and D. Liu, ''The application of ant colony optimization algorithm in the flight landing scheduling problem,'' *Appl. Mech. Mater.*, vols. 411–414, pp. 2698–2703, Sep. 2013, doi: 10.4028/www.scientific.net/amm.411-414.2698.

[10] J. Fairbrother, K. G. Zografos, and K. D. Glazebrook, ''A slot-scheduling mechanism at congested airports that incorporates efficiency, fairness, and airline preferences,'' *Transp. Sci.*, vol. 54, no. 1, pp. 115–138, Jan. 2020, doi: 10.1287/trsc.2019.0926.

[11] W. Wu, Z. Lin, and M. Wei, ''Research on a collaborative evolution model of multi-airport route network considering subsidy strategies,'' *Math. Biosci. Eng.*, vol. 20, no. 11, pp. 19808–19838, 2023, doi: 10.3934/mbe.2023877.

[12] R. Yang, Z. Feng, L. Meilong, H. Zhang, and M. Ji, ''Genetic-algorithm-based approaches for enhancing fairness and efficiency in dynamic airport slot allocation,'' *Chin. J. Aeronaut.*, vol. 38, no. 8, Aug. 2025, Art. no. 103634, doi: 10.1016/j.cja.2025.103634.

[13] K. N. Androutsopoulos and M. A. Madas, ''Being fair or efficient? A fairness-driven modeling extension to the strategic airport slot scheduling problem,'' *Transp. Res. E, Logistics Transp. Rev.*, vol. 130, pp. 37–60, Oct. 2019, doi: 10.1016/j.tre.2019.08.010.

[14] Y. Jiang and K. G. Zografos, ''A decision making framework for incorporating fairness in allocating slots at capacity-constrained airports,'' *Transp. Res. C, Emerg. Technol.*, vol. 126, May 2021, Art. no. 103039, doi: 10.1016/j.trc.2021.103039.

[15] S. Varenna, H. Li, M. Ribeiro, and B. F. Santos, ''Stochastic discrete event simulation of airline network and maintenance operations,'' *J. Air Transp. Manage.*, vol. 125, May 2025, Art. no. 102789, doi: 10.1016/j.jairtraman.2025.102789.

[16] W. Tian, M. Yang, X. Zhou, J. Yin, and X. Shi, ''Airport field path optimization method based on conflict hotspot avoidance mechanism,'' *Appl. Sci.*, vol. 15, no. 15, p. 8204, 8204, doi: 10.3390/app15158204.

[17] T. Chen and C. Guestrin, ''XGBoost: A scalable tree boosting system,'' in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[18] A. Attar, M. Babaee, S. Raissi, and M. Nojavan, ''Airside optimization framework covering multiple operations in civil airport systems with a variety of aircraft: A simulation-based digital twin,'' *Systems*, vol. 12, no. 10, p. 394, Sep. 2024, doi: 10.3390/systems12100394.

[19] H. Liu, S. Li, F. Sun, W. Fan, W.-H. Ip, and K.-L. Yung, "Adaptive dynamic programming with reinforcement learning on optimization of flight departure scheduling," *Aerospace*, vol. 11, no. 9, p. 754, Sep. 2024, doi: 10.3390/aerospace11090754.
[20] J. Ouyang, C. Zhu, X. Tang, and J. Zhang, "Multi-stand grouped operations method in airport bay area based on deep reinforcement learning," *Aerospace*, vol. 12, no. 5, p. 398, Apr. 2025, doi: 10.3390/aerospace12050398.
[21] F. Wang, M. Zhou, Y. Xing, H.-W. Wang, Y. Peng, and Z. Chen, "A deep reinforcement learning framework for multi-fleet scheduling and optimization of hybrid ground support equipment vehicles in airport operations," *Appl. Sci.*, vol. 15, no. 17, p. 9777, Sep. 2025, doi: 10.3390/app15179777.
[22] K. Ramamoorthy and G. Hunter, "Simulation-based airport capacity estimation," in *Proc. Aviation Technol., Integr., Oper. Conf.*, Aug. 2013, Paper AIAA 2013-4354, doi: 10.2514/6.2013-4354.
[23] L. H. Lee, H. C. Huang, C. Lee, E. P. Chew, W. Jaruphongsa, Y. Y. Yong, Z. Liang, C. H. Leong, Y. P. Tan, K. Namburi, E. Johnson, and J. Banks, "Discrete event simulation model for airline operations: SIMAIR," in *Proc. Int. Conf. Mach. Learn. Cybern.*, New Orleans, LA, USA, Dec. 2003, pp. 1656–1662, doi: 10.1109/WSC.2003.1261616.
[24] Y. Li and Y. Liu, "Multi-airport system flight slot optimization method based on absolute fairness," *Math. Biosci. Eng.*, vol. 20, no. 10, pp. 17919–17948, 2023, doi: 10.3934/mbe.2023797.
[25] R. Yang, M. Le, and Q. Wang, "Multi-objective airport slot allocation with demand-side fairness considerations," *Aerospace*, vol. 12, no. 2, p. 119, Feb. 2025, doi: 10.3390/aerospace12020119.
[26] H. Li, X. Wu, M. Ribeiro, B. Santos, and P. Zheng, "Deep reinforcement learning approach for real-time airport gate assignment," *Oper. Res. Perspect.*, vol. 14, Jun. 2025, Art. no. 100338, doi: 10.1016/j.orp.2025.100338.
[27] K. Liu, F. Hu, H. Lin, X. Cheng, J. Chen, J. Song, S. Feng, G. Su, and C. Zhu, "Deep reinforcement learning for real-time ground delay program revision and corresponding flight delay assignments," in *Proc. IEEE 27th Int. Conf. Intell. Transp. Syst. (ITSC)*, Edmonton, AB, Canada, Sep. 2024, pp. 1302–1307, doi: 10.1109/ITSC58415.2024.10920242.
[28] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," 1998, *arXiv:cs/9809099*.
[29] S. Keshav, "A control-theoretic approach to flow control," in *Proc. Conf. Commun. Archit. Protocols*, Aug. 1991, pp. 3–15, doi: 10.1145/115992.115995.
[30] M. Tao, L. Ma, and Y. Ma, "Flight schedule adjustment for hub airports using multi-objective optimization," *J. Intell. Syst.*, vol. 30, no. 1, pp. 931–946, Jul. 2021, doi: 10.1515/jisys-2020-0114.
[31] N. Ivanov, F. Netjasov, R. Jovanović, S. Starita, and A. Strauss, "Air traffic flow management slot allocation to minimize propagated delay and improve airport slot adherence," *Transp. Res. A, Policy Pract.*, vol. 95, pp. 183–197, Jan. 2017, doi: 10.1016/j.tra.2016.11.010.
[32] M. K. Shambour and M. A. Abu-Hashem, "Optimizing airport slot scheduling problem using optimization algorithms," *Soft Comput.*, vol. 27, no. 12, pp. 7939–7955, Jun. 2023, doi: 10.1007/s00500-023-07987-3.
[33] K. A. Lundaeva, Z. A. Saranin, K. N. Pospelov, and A. M. Gintciak, "Demand forecasting model for airline flights based on historical passenger flow data," *Appl. Sci.*, vol. 14, p. 11413, Dec. 2024, doi: 10.3390/app142311413.
[34] J. Zhao, Y. Nie, S. Ni, and X. Sun, "Traffic data imputation and prediction: An efficient realization of deep learning," *IEEE Access*, vol. 8, pp. 46713–46722, 2020, doi: 10.1109/ACCESS.2020.2978530.
[35] D. Bertsimas and J. Dunn, *Machine Learning Under a Modern Optimization Lens*. Waltham, MA, USA: Dynamic Ideas LLC, 2019.
[36] E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias, "Short-term traffic forecasting: Where we are and where we're going," *Transp. Res. C, Emerg. Technol.*, vol. 43, pp. 3–19, Jun. 2014, doi: 10.1016/j.trc.2014.01.005.
[37] J. E. Beasley, M. Krishnamoorthy, Y. M. Sharaiha, and D. Abramson, "Scheduling aircraft landings—The static case," *Transp. Sci.*, vol. 34, no. 2, pp. 180–197, May 2000, doi: 10.1287/trsc.34.2.180.12302.
[38] A. A. Kablan, H. R. K. Elberkawi, and E. A. Eldharif, "Airline scheduling model using genetic algorithm," in *Proc. IEEE 4th Int. Maghreb Meeting Conf. Sci. Techn. Autom. Control Comput. Eng. (MI-STA)*, Tripoli, Libya, May 2024, pp. 718–722, doi: 10.1109/MI-STA61267.2024.10599636.
[39] S. Liu, S. Wang, M. Hu, and L. Yang, "Slot optimization based on coupled airspace capacity of multi-airport system," *Appl. Sci.*, vol. 15, no. 12, p. 6759, Jun. 2025, doi: 10.3390/app15126759.



SAAD TALAL ALHARBI received the B.Sc. degree in computer science from Taibah University, Madinah, Saudi Arabia, in 2004, and the M.Sc. degree (Hons.) in software engineering and the Ph.D. degree in computer science (human–computer interaction) from the University of Bradford, U.K., in 2006 and 2009, respectively.

He is currently a Full Professor with the College of Computer Science and Engineering, Taibah University. Previously, he was the Dean of the Information Technology Deanship, Taibah University, and has held several academic and administrative leadership positions, including the Head of the Computer Science Department and the Vice Dean of the Information Technology Deanship. During his tenure, he supervised and led several institutional information technology projects, including enterprise resource planning systems, learning management systems, and digital infrastructure initiatives at the university level. His research interests include artificial intelligence-driven optimization, machine learning-guided decision systems, intelligent scheduling and resource allocation, human–computer interaction, and data-driven systems for smart and intelligent environments. His work spans both theoretical and applied research, with applications in transportation systems, smart infrastructure, and intelligent decision-support systems.